



**SWAMINARAYAN
UNIVERSITY**
THE RESERVOIR OF ALL KNOWLEDGE

**Faculty of Engineering (Degree)
Semester – 7th**

Subject Name – Information security

Subject Code – BE103707

Faculty Name – Radhika Gokani

CHAPTER -4

1. Public Key Cryptosystems with Applications

Public key cryptosystems, also known as asymmetric cryptography, play a vital role in information security. This system uses a pair of keys: a public key and a private key, which are mathematically linked but distinct in their functions.

Components:

- **Public Key:** This key is openly shared and used to encrypt data or verify transactions. It's derived from the private key using complex mathematical algorithms, making it safe to distribute without compromising security.
- **Private Key:** This key is kept confidential and used to decrypt data or sign transactions. It's essential to store private keys securely to prevent unauthorized access.

How Public Key Cryptography Works:

1. A sender encrypts data using the recipient's public key.
2. The encrypted data can only be decrypted by the recipient's private key.
3. Digital signatures are created using the sender's private key and verified by the recipient using the sender's public key.

Applications of Public Key Cryptography:

- a. **Secure Web Browsing (HTTPS):** Establishes a secure connection between a web browser and server.
- b. **Email Encryption (PGP/GPG):** Ensures confidentiality and authenticity of emails.
- c. **Digital Signatures:** Verifies the authenticity and integrity of digital documents and software.
- d. **Blockchain and Cryptocurrencies:** Secures transactions and protects user identities.
- e. **Secure Shell (SSH):** Enables secure remote access to servers .

Benefits:

- **Secure Communication:** Enables secure data exchange without prior secret key exchange.
- **Digital Signatures:** Provides authentication and non-repudiation.

- **Scalability:** Easier to manage key distribution in large networks.

Challenges:

Computational Overhead: More computationally intensive than symmetric key algorithms.

Vulnerability to Quantum Computing: Some public key algorithms are vulnerable to quantum computer attacks.

Key Management: Secure storage and handling of private keys are important.

2. Requirements and Cryptanalysis

Public key cryptosystems have specific requirements to ensure their security and effectiveness.

Public-Key Cryptography Requirements

Computational Infeasibility: It should be computationally infeasible for an adversary to determine the private key from the public key.

Easy Encryption and Decryption: It should be easy for the sender to encrypt the message using the public key and for the receiver to decrypt it using the private key.

Trapdoor One-Way Function: The encryption algorithm should be a trapdoor one-way function, meaning it's easy to compute in one direction but hard to reverse without the private key.

Applications of Public-Key Cryptosystems

1. Encryption/Decryption: Securely sending messages between parties.
2. Digital Signatures: Authenticating the sender and ensuring data integrity.
3. Key Exchange: Securely exchanging cryptographic keys between parties.

Public-Key Cryptanalysis

Public-key cryptosystems are vulnerable to certain types of attacks, including:

1. **Brute-Force Attack:** Exhaustively searching for the private key.
2. **Probable-Message Attack:** Exploiting known or probable message contents.
3. **Key Recovery Attack:** Finding the private key given the public key.

To mitigate these risks, public-key cryptosystems rely on large key sizes and complex mathematical algorithms, such as RSA and elliptic curve cryptography (ECC). ECC, in particular, offers better security with smaller key sizes, making it more efficient.

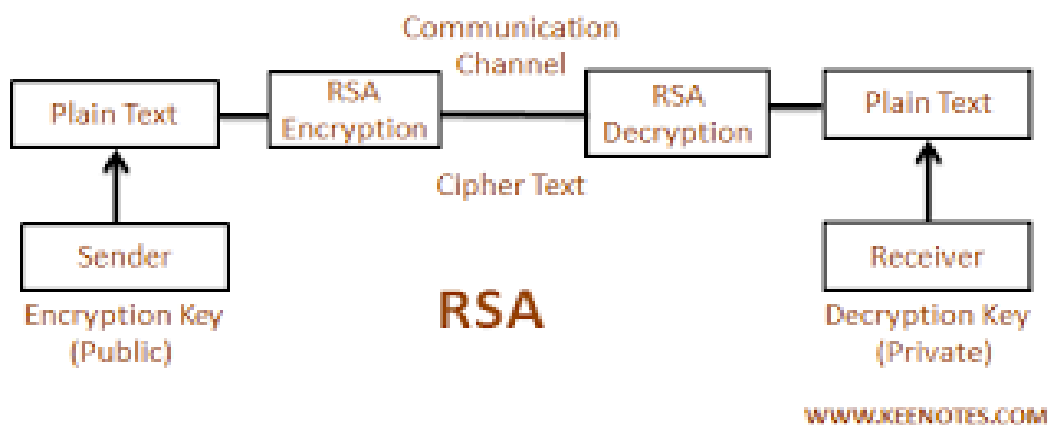
4. RSA algorithm

RSA (Rivest–Shamir–Adleman) is one of the most widely used **asymmetric cryptographic algorithms**, meaning it uses **two keys**: a **public key** for encryption and a **private key** for decryption. It's foundational to secure communication on the internet, including HTTPS, digital signatures, and secure email.

Algorithm 1 The structure of RSA algorithm as follows.

- 1: **Input Values:** p and q
 - 2: **Compute:**
 - 3: $n = p \times q$
 - 4: $\phi(n) = (p-1)(q-1)$
 - 5: **Select Integer values:** e [(gcd(), e) = 1; $1 < e < \phi(n)$]
 - 6: **Compute:** d de mod $\phi(n) = 1$
 - 7: $C = M^e \bmod n$
 - 8: **Encryption:** $M < n$ $C = M \bmod n$
 - 9: **Decryption:** $CM = C \bmod n$
-

RSA is based on the mathematical difficulty of **factoring large prime numbers**. Here's a simplified breakdown:



1. Key Generation

- Choose two large prime numbers: p and q
- Compute $n = p \times q$ (used as modulus for both keys)
- Compute Euler's totient: $\phi(n) = (p-1)(q-1)$
- Choose an encryption exponent e such that $1 < e < \phi(n)$ and $\gcd(e, \phi(n)) = 1$
- Compute the decryption exponent d such that $(d \times e) \bmod \phi(n) = 1$

Public Key: (n, e) **Private Key:** (n, d)

2. Encryption

To encrypt a message M:

Code

$$C = M^e \bmod n$$

Where C is the ciphertext.

3. Decryption

To decrypt the ciphertext C:

Code

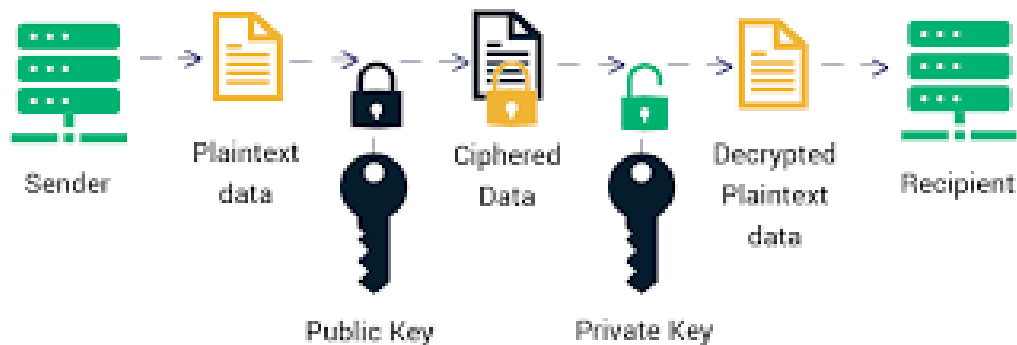
$$M = C^d \bmod n$$

Where M is the original message.

Applications

- **Digital Signatures:** Verifying authenticity and integrity
- **Secure Data Transmission:** Used in SSL/TLS protocols
- **Key Exchange:** Often used to securely share symmetric keys

How RSA Encryption Works



Example

Let's say:

- $p = 61, q = 53 \rightarrow n = 3233$
- $\phi(n) = 3120$
- Choose $e = 17$
- Compute $d = 2753$

To encrypt the ASCII value of "H" (72):

Code

$$C = 72^{17} \bmod 3233 = 2103$$

To decrypt:

Code

$$M = 2103^{2753} \bmod 3233 = 72$$

So, the message "H" is successfully recovered.

5. Its computational aspects and security

Computational Aspects of RSA

RSA's strength lies in number theory, particularly the difficulty of **factoring large integers**. Here's what makes it computationally intense:

1. Key Generation

- Involves selecting two large prime numbers (hundreds or thousands of bits long).
- Computing $n = p \times q$ and Euler's totient $\phi(n) = (p-1)(q-1)$ is straightforward.
- Finding e and d (modular inverses) requires algorithms like the **Extended Euclidean Algorithm**.

2. Encryption & Decryption

- Both operations use **modular exponentiation**, which is computationally expensive for large keys.
- Efficient algorithms like **square-and-multiply** are used to speed up exponentiation.

3. Performance

- RSA is slower than symmetric algorithms (like AES).
- It's typically used to encrypt small data (e.g., keys), not large files.
- For speed, RSA is often combined with symmetric encryption in hybrid systems.

Security Aspects of RSA

RSA's security is based on the **RSA problem**: given n and e , it's computationally infeasible to determine d without factoring n .

Key Security Factors:

- **Difficulty of Prime Factorization:** Factoring a large n (e.g., 2048-bit) into p and q is not feasible with current algorithms and computing power.
- **Key Size:** Larger keys (2048-bit or more) offer stronger security. As key size increases, the time to factor n grows exponentially.
- **Modular Arithmetic:** RSA relies on properties like Euler's theorem and multiplicative inverses, which are mathematically sound and hard to reverse without the private key.

Known Vulnerabilities:

- **Small Key Sizes:** Keys under 1024 bits are vulnerable to brute-force attacks.
- **Poor Randomness:** If p and q aren't truly random or are reused, RSA can be broken.
- **Side-Channel Attacks:** Timing or power analysis can leak information about d .

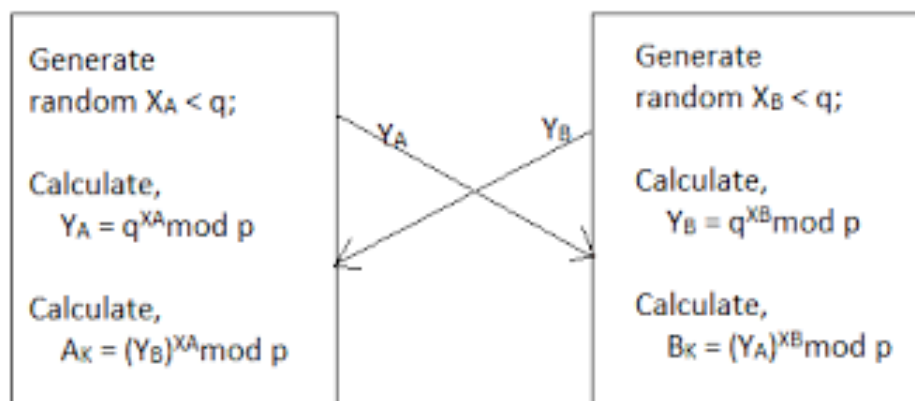
Note

RSA is secure **if implemented correctly** with large keys and good randomness. But it's computationally heavy, which is why it's often used for **key exchange** rather than bulk data encryption.

6. Diffie-Hellman Key Exchange algorithm

The **Diffie-Hellman (DH) key exchange** is a foundational cryptographic protocol that allows two parties to securely establish a **shared secret key** over an **insecure channel** without ever transmitting the key itself. This shared key can then be used for symmetric encryption.

The brilliance of Diffie-Hellman lies in **modular arithmetic** and the difficulty of solving the **discrete logarithm problem**. Even if someone intercepts the exchanged data, they can't feasibly compute the shared secret.



Step-by-Step Breakdown

Let's say Alice and Bob want to communicate securely:

1. Public Parameters

- Choose a large prime number p
- Choose a primitive root g (also called a generator) of p

These values are public and can be shared openly.

2. Private Keys

- Alice chooses a secret integer a

- Bob chooses a secret integer b

3. Compute Public Keys

- Alice computes $A = g^a \bmod p$
- Bob computes $B = g^b \bmod p$

They exchange A and B over the public channel.

4. Compute Shared Secret

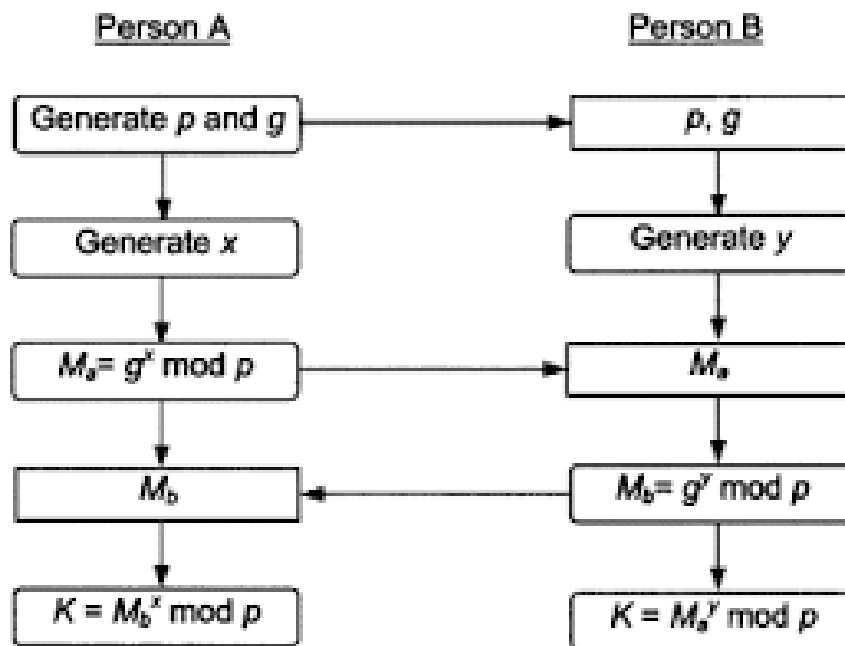
- Alice computes $s = B^a \bmod p$
- Bob computes $s = A^b \bmod p$

Both arrive at the same shared secret s , because:

Code

$$B^a \bmod p = (g^b)^a \bmod p = g^{(ab)} \bmod p$$

$$A^b \bmod p = (g^a)^b \bmod p = g^{(ab)} \bmod p$$



Security Aspects

- **Discrete Logarithm Problem:** Given g , p , and $g^a \bmod p$, it's computationally hard to find a .
- **No Key Transmission:** The actual shared key is never sent—only derived.
- **Vulnerabilities:**
 - Susceptible to **man-in-the-middle attacks** if authentication isn't used.
 - Weak parameters (small p or poor g) can compromise security.
 - Not quantum-resistant—quantum computers could break DH using Shor's algorithm.

Example

Let's use small numbers for clarity:

- Public: $p = 23$, $g = 5$
- Alice's secret: $a = 6 \rightarrow A = 5^6 \bmod 23 = 8$
- Bob's secret: $b = 15 \rightarrow B = 5^{15} \bmod 23 = 2$
- Shared secret:
 - Alice: $s = 2^6 \bmod 23 = 9$
 - Bob: $s = 8^{15} \bmod 23 = 9$

Both now share the secret 9.

7. **Man-in-Middle attack**

A **Man-in-the-Middle attack** is a cyberattack where an attacker secretly intercepts and possibly alters the communication between two parties who believe they are directly communicating with each other. It’s like someone eavesdropping on a private conversation and occasionally whispering false information to both sides.

Work

Imagine Alice wants to send a message to Bob. Mallory (the attacker) inserts herself between them:

1. Alice sends a message to Bob.
2. Mallory intercepts it, reads or modifies it.
3. Mallory forwards the altered message to Bob.
4. Bob replies, thinking he’s talking to Alice.
5. Mallory intercepts Bob’s reply and sends it to Alice.

Neither Alice nor Bob realizes Mallory is in the middle.

Common Techniques

Technique	Description
ARP Spoofing	Attacker tricks devices into sending data to them by faking MAC addresses.
DNS Spoofing	Redirects users to malicious websites by corrupting DNS responses.
SSL Stripping	Downgrades secure HTTPS connections to HTTP to intercept data.
Wi-Fi Eavesdropping	Attacker sets up rogue Wi-Fi hotspots to intercept traffic.
Session Hijacking	Steals session tokens to impersonate users.

How to Prevent It

- Use **end-to-end encryption** (like HTTPS/TLS)
- Verify **SSL certificates** before entering sensitive info
- Avoid **public Wi-Fi** for financial or private transactions
- Use **VPNs** to encrypt traffic
- Implement **multi-factor authentication**
- Keep software and browsers **up to date**

Real-World Example

A classic MITM scenario: You're on public Wi-Fi at a café. An attacker on the same network uses ARP spoofing to redirect your traffic through their device. You log into your bank account, and they capture your credentials—even if the site looks secure.
